

TRON — Multi-Sig Crypto Wallet

A self-hosted, multi-signature wallet application for the TRON blockchain, with a Node.js backend for building, co-signing, and broadcasting transactions, and built-in tooling for managing Bandwidth/Energy fees so transfers don't fail when the wallet is low on TRX.

Table of Contents

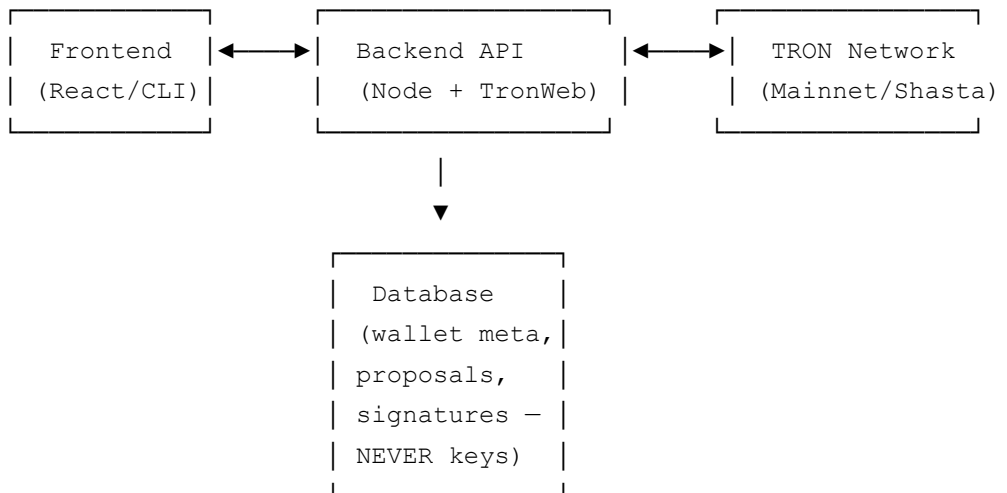
1. [Overview](#)
 2. [Architecture](#)
 3. [Tech Stack](#)
 4. [Repo Layout](#)
 5. [Prerequisites](#)
 6. [Getting Started](#)
 7. [Environment Variables](#)
 8. [Multi-Sig Design](#)
 9. [Handling Low TRX / Fee Failures](#)
 10. [Transparent Fee Sponsorship \(Sponsor Account\)](#)
 11. [Backend API](#)
 12. [Testing on Testnet](#)
 13. [GitHub Workflow](#)
 14. [CI/CD](#)
 15. [Security Checklist](#)
 16. [Roadmap](#)
 17. [License](#)
-

Overview

This app lets a group of signers jointly control a TRON wallet. No single key can move funds — transactions are proposed, signed by a threshold of participants, and only then broadcast. It supports:

- Creating/importing a TRON multi-signature account
 - Proposing transfers (TRX and TRC-20 tokens)
 - Collecting signatures from co-signers until the threshold is met
 - Broadcasting the fully-signed transaction
 - Monitoring and topping up Bandwidth/Energy so transactions don't get stuck when the wallet's TRX balance is low
-

Architecture



The backend never stores private keys. Signing happens client-side (browser extension, hardware wallet, or a signer's own machine) — the backend only coordinates proposals, collects signatures, and relays the final broadcast.

Tech Stack

Layer	Choice

Blockchain SDK	TronWeb
Backend	Node.js + Express (or Fastify)
Database	PostgreSQL (proposals, signature status, wallet metadata)
Auth	JWT / signed-message auth per signer
Frontend	React (optional — API-first design)
Infra	Docker + docker-compose for local dev
CI/CD	GitHub Actions

Repo Layout

```

tron-wallet/
├── backend/
│   ├── src/
│   │   ├── controllers/           # route handlers
│   │   ├── services/
│   │   │   ├── tron.service.js    # TronWeb wrapper
│   │   │   ├── multisig.service.js # proposal/signature logic
│   │   │   └── resources.service.js # bandwidth/energy management
│   │   ├── models/               # DB schemas
│   │   ├── routes/
│   │   └── app.js
│   ├── test/
│   ├── .env.example
│   └── package.json
├── frontend/                     # optional
├── docs/
│   ├── MULTISIG.md
│   └── DEPLOYMENT.md
├── .github/
│   ├── workflows/ci.yml
│   └── ISSUE_TEMPLATE/
├── docker-compose.yml
└── README.md

```

Prerequisites

- Node.js 18+ and npm/yarn
 - Docker (optional, for local Postgres)
 - A GitHub account and `git` installed
 - TronLink or another TRON-compatible signer for testing multi-sig locally
 - Testnet TRX from the [Shasta Faucet](#) or [Nile Faucet](#)
-

Getting Started

```
# 1. Clone the repo
git clone https://github.com/<your-org>/tron-wallet.git
cd tron-wallet

# 2. Install dependencies
cd backend
npm install

# 3. Copy and configure environment variables
cp .env.example .env
# edit .env with your TRON node URL, API keys, and DB connection string

# 4. Start a local Postgres instance (or use docker-compose)
docker-compose up -d db

# 5. Run migrations
npm run migrate

# 6. Start the backend in dev mode
npm run dev
```

The API will be available at `http://localhost:4000` by default.

Environment Variables

Create `backend/.env` from `.env.example`:

```
# TRON node / API
TRON_FULL_HOST=https://api.shasta.trongrid.io # use https://api.trongrid.io for
TRON_API_KEY=your_trongrid_api_key
```

```
# Network selection
TRON_NETWORK=shasta    # shasta | nile | mainnet

# Database
DATABASE_URL=postgres://user:password@localhost:5432/tron_wallet

# Auth
JWT_SECRET=replace_with_a_long_random_string

# Multisig defaults
DEFAULT_SIGNATURE_THRESHOLD=2
DEFAULT_SIGNER_COUNT=3

# Fee / resource management
MIN_TRX_RESERVE=50      # minimum TRX to keep unfrozen for fallback fees
FEE_LIMIT_SUN=100000000 # max fee limit for contract calls (in SUN)
AUTO_STAKE_FOR_ENERGY=true # see "Handling Low TRX" below
```

Never commit `.env` — it's already in `.gitignore`.

Multi-Sig Design

TRON has **native account-level multi-signature support** — you don't need a custom smart contract for basic multi-sig (though you can layer one on top for more complex approval logic).

Each TRON account has three permission groups, each with its own weighted signer list and threshold:

- `owner` — can change account permissions itself (highest sensitivity)
- `active` — can send transactions, invoke contracts, vote
- `witness` — for super representative operations (usually unused here)

Setup flow

1. **Create/choose the wallet account.**
2. **Update its permissions** with `updateAccountPermission` to add signers and set a weighted threshold, e.g. 3 signers, weight 1 each, threshold 2 (a 2-of-3 wallet).
3. **Every outgoing transaction** must then be signed by signers whose combined weight $\geq$ threshold before it's valid on-chain.

Backend responsibilities

- `POST /wallets/:id/proposals` — build an unsigned transaction, store it as a pending proposal
- `POST /proposals/:id/sign` — accept a signer's signature (produced client-side) and attach it via `tronWeb.trx.multiSign`
- Once signature count/weight meets the threshold, backend calls `tronWeb.trx.sendRawTransaction` to broadcast

See `docs/MULTISIG.md` for the full permission JSON schema and example TronWeb calls.

Handling Low TRX / Fee Failures

TRON doesn't use simple "gas price" fees like Ethereum — it uses two resources, **Bandwidth** and **Energy**, and this is almost always the cause of "stuck on fees" issues:

Resource	Used for	How to get it
Bandwidth	Regular transfers (TRX, TRC-10)	Free daily allotment (~5,000 points/account) + staking TRX
Energy	Smart contract calls (TRC-20 transfers, multisig ops)	Staking TRX for Energy, or delegation

If an account runs out of both free allocation and staked resources, TRON falls back to **burning TRX directly** to pay for bandwidth/energy — and if the wallet's TRX balance is too low to cover that burn, the transaction fails with an "insufficient balance" or "bandwidth" error.

Strategies implemented in `resources.service.js`

1. **Stake TRX for Energy/Bandwidth (Stake 2.0)** — freeze a portion of TRX using `tronWeb.trx.freezeBalanceV2()` targeted at `ENERGY` or `BANDWIDTH`. This is the standard fix: a small staked amount covers ongoing operational fees without burning TRX on every transaction.

```
await tronWeb.trx.freezeBalanceV2(amountInSun, 'ENERGY');
```

2. **Resource delegation** — if the wallet itself can't stake enough, a separate "sponsor" account can delegate Energy/Bandwidth to it via `delegateResource`, without

transferring TRX ownership.

```
await tronWeb.trx.delegateResource(amountInSun, receiverAddress, 'ENERGY');
```

3. **Set an explicit `feeLimit`** on every contract call (TRC-20 transfers, multisig permission updates) so failed/underfunded calls fail fast and cheap instead of silently consuming the whole reserve.
4. **Pre-flight balance/resource check** — before building a proposal, the backend calls `tronWeb.trx.getAccountResources(address)` and compares available Bandwidth/Energy + TRX balance against the estimated cost. If insufficient, the API returns a clear `INSUFFICIENT_RESOURCES` error with the shortfall amount, instead of letting the transaction fail on-chain.
5. **`MIN_TRX_RESERVE`** — the backend warns (and can block outgoing proposals) once the wallet's liquid TRX drops below this threshold, so you're never caught without a fallback for the burn-TRX path.

Quick manual fix while developing

If you're just stuck locally:

```
# check current resources
curl -X POST https://api.shasta.trongrid.io/wallet/getaccountresource \
  -d '{"address":"<your_address_base58>"}'

# get more testnet TRX
# Shasta: https://www.trongrid.io/shasta
# Nile:   https://nileex.io/join/getJoinPage
```

On mainnet, buying/staking a modest amount of TRX (or renting Energy from a resource marketplace) resolves this permanently — no code changes needed.

Transparent Fee Sponsorship (Sponsor Account)

For onboarding flows where a new/under-funded wallet needs help covering its first transfers, the correct pattern is **resource delegation from a sponsor account you control** — never a “pay a fee to this address” prompt routed to the end user. The distinction matters:



Not implemented (and won't be)



Implemented here

App tells the user "insufficient TRX for fees, send X TRX to address Y to unlock your transfer"	App delegates Bandwidth/Energy to the user's wallet from a sponsor account <i>you</i> operate
Funds move from user → app-controlled address	No funds move from the user at all — a resource, not TRX, is lent
Destination address is hidden/unexplained	Sponsor relationship and cost (if any) are shown in the UI before confirmation
Irreversible once sent	Delegated resources can be reclaimed (<code>undelegateResource</code>) and simply expire if not renewed

How it works

1. Your organization funds and stakes TRX into a **sponsor account** (a wallet you own, separate from user wallets).
2. When a user's wallet lacks enough Bandwidth/Energy for a transfer, the backend — with the sponsorship clearly disclosed in the UI/API response — delegates the needed resource from the sponsor account directly to the user's address using `delegateResource`. This does **not** transfer TRX ownership; it only lends transaction capacity, and can be revoked.
3. The user's transfer then succeeds using the delegated resource. If you charge for this service, the cost is shown up front in fiat/TRX terms and charged through your normal billing (Stripe, in-app credits, etc.) — never as a disguised on-chain "fee" transfer.

`resources.service.js` additions

```
// Delegate Energy/Bandwidth from the sponsor account to a user's wallet.
// Read-only on the user's TRX balance — no funds are moved from the user.
async function sponsorDelegate({ toAddress, resourceType, amountSun }) {
  const tx = await tronWeb.trx.delegateResource(
    amountSun,
    toAddress,
    resourceType,          // 'ENERGY' | 'BANDWIDTH'
    SPONSOR_ACCOUNT_ADDRESS, // from env: SPONSOR_ACCOUNT_ADDRESS
    false,                 // lock (optional time-lock)
  );
  await db.sponsorships.create({ toAddress, resourceType, amountSun, txId: tx.txid });
  return tx;
}
```



```
// Reclaim resources once no longer needed (e.g., after a grace period).
async function sponsorUndelegate({ toAddress, resourceType, amountSun }) {
    return tronWeb.trx.undelegateResource(amountSun, toAddress, resourceType, SPONS
}

// One-click SELF-staking: the user's own TRX funds their own future fees.
// This is the default path – sponsorship is only a fallback for new/empty wallet
async function selfStake({ ownerAddress, amountSun, resourceType }) {
    return tronWeb.trx.freezeBalanceV2(amountSun, resourceType, ownerAddress);
}
```

Additional environment variables

```
# Sponsor account (used ONLY to delegate Bandwidth/Energy, never to receive user
SPONSOR_ACCOUNT_ADDRESS=T...your_sponsor_wallet
SPONSOR_ACCOUNT_PRIVATE_KEY= # store in a secrets manager, not .env in producti
SPONSOR_DELEGATION_LIMIT_SUN= # cap per-wallet delegation to control sponsor exp
SPONSOR_GRACE_PERIOD_HOURS=24 # auto-undelegate after this window if unused
```

UI requirement

Any screen offering sponsored resources must state, in plain language: *“You don’t have enough Energy/Bandwidth for this transfer. We can lend you enough to complete it at no cost to your balance — no TRX will be taken from your wallet.”* Never present this as a fee owed by the user to a third-party address.

Backend API

Method	Endpoint	Description
POST	/wallets	Create a new multisig wallet
GET	/wallets/:address	Get wallet info + balances
GET	/wallets/:address/resources	Get Bandwidth/Energy status
POST	/wallets/:address/stake	Freeze TRX for Bandwidth/Energy (self-funded)
POST	/wallets/:address/sponsor-request	Request delegated Energy/Bandwidth from the sponsor account (discloses terms, no funds move from user)

DELETE	/wallets/:address/sponsor-request	Revoke/reclaim a sponsored delegation
POST	/proposals	Create a transfer proposal
POST	/proposals/:id/sign	Submit a signer's signature
GET	/proposals/:id	Check signature progress
POST	/proposals/:id/broadcast	Broadcast once threshold is met
GET	/health	Health check

Testing on Testnet

1. Set `TRON_NETWORK=shasta` (or `nile`) in `.env`.
2. Fund your wallet addresses from the relevant faucet.
3. Run the integration suite:

```
npm run test:integration
```

4. Manually exercise the multisig flow with 2–3 test accounts before ever pointing at mainnet.

GitHub Workflow

- **Branching:** `main` (protected) $\leftarrow$ `develop` $\leftarrow$ feature branches (`feat/...`, `fix/...`)
- **Commits:** [Conventional Commits](#) (`feat:`, `fix:`, `chore:`, `docs:`)
- **Pull requests:** required for all merges to `develop` / `main` ; at least one review + passing CI
- **Issues:** use the templates in `.github/ISSUE_TEMPLATE/` (bug report, feature request)
- **Releases:** tag `vX.Y.Z` on `main` ; GitHub Actions builds and publishes release notes

Suggested first steps in your repo:

```
git checkout -b feat/multisig-service
```

```
# ...make changes...
git add .
git commit -m "feat: add multisig proposal/signature service"
git push origin feat/multisig-service
# open a PR into develop
```

CI/CD

`.github/workflows/ci.yml` should run on every PR:

- `npm ci`
- Lint (`eslint`)
- Unit tests
- Integration tests against Shasta testnet (using repo secrets for faucet keys — never commit real keys)

Add TRON credentials as **GitHub Actions secrets**, not in the repo: `Settings` → `Secrets` and `variables` → `Actions`.

Security Checklist

- Private keys never touch the backend or the database
 - `.env` is git-ignored; secrets live in a vault or GitHub Actions secrets
 - Multisig threshold is ≥ 2 -of-3 for any wallet holding real funds
 - `feeLimit` is set explicitly on every contract call
 - Rate limiting and auth on all proposal/sign endpoints
 - Full test coverage on Shasta/Nile before touching mainnet
 - Independent code/security audit before mainnet deployment with real funds
 - Admin/owner permissions are never a single unprotected key
-

Roadmap

- TRC-20 token support in the transfer proposal flow
 - Resource marketplace integration (auto-rent Energy when reserves run low)
 - Hardware wallet (Ledger) signer support
 - Notification webhooks for pending signatures
 - Optional smart-contract-based approval layer (for time-locks, spend limits)
-

License

MIT — see [LICENSE](#).